```
  if edge != null and edgeSuffix == “”
then                                        #7
    result ← longestPrefix(edge.destination, sSuffix,
prefix+commonPrefix)
  if result != null
then                                                    #8
    return result
  elsif node.keyNode
then                                                   #9
    return prefix
  else
    return
null
    #10
```

#1 Method `longestPrefix` takes a trie node, the string key `s` to be searched, and a string with the path from root to `node`; it returns the longest prefix of `s` stored in the trie.
#2 Check if the string searched is empty: if it is, since this method is implemented using recursion, we know it has traversed the whole path in the trie.
#3 We are at the final node of the path for the searched string: if it’s a *key node*, then the string itself (by know accumulated into `prefix`) is its longest prefix in the trie.
#4 Otherwise, we need to return `null` to let the caller know we haven’t found any key yet.
#5 Since `s` is not the empty string, we can see if there is an edge matching it, even partially.
#6 Initialize this temporary variable to `null`. In case we can traverse another edge, it will hold the result of the recursive call.
#7 Match case #1 (WRT figure 6.15). There is an edge whose label is a prefix to `s`: we just need to traverse the edge